PROJECT REVIEW 3

AuxFlow: Static Analysis for Multi-Pass Document Compilation

1. Title Page

Project title	AuxFlow: Static Analysis for Multi-Pass Document Compilation
Student name	Upayan Mazumder
Register number	24BDS0367
Programme	B.Tech CSE (Data Science)
Department	School of Computer Science and Engineering (SCOPE)
College	Vellore Institute of Technology, Vellore, Tamil Nadu, India
Academic year	2026-2027
Project guide	Banupriya R
Review	Review 3: analysis, planning, verification, and evaluation

2. Abstract

A LaTeX document with cross-references, a table of contents, or a bibliography needs several engine runs, because each run reads the auxiliary files that the previous run wrote. Current tools find the run count by running the engine and stopping at a fixed limit. Some documents never settle, and these tools find that out only after they reach the limit.

AuxFlow is a compiler front end and static analyzer for a declared subset of LaTeX. It lowers a document to a graph of auxiliary fact instances with lagged edges, finds the facts that control how much content the document generates, and classifies the document as CERTIFIED, BENIGN_CYCLIC, AMPLIFYING, or UNKNOWN before any engine run. A planner turns the class into a build plan, and a runtime verifier checks the plan against the real engine.

On a 15-document corpus, 7 baselines, and 10 repetitions (1,050 runs), all 7 certified documents received a bound at least as large as the measured run count. All 6 non-convergent documents were flagged before the engine started. Static analysis took 1.3 ms at the median and 10.4 ms at most. A guard added after the first measurement changed three declared false negatives into UNKNOWN results. The report states this change in section 7.

3. Introduction

Background

LaTeX resolves a reference in two steps. One engine run writes the label value to the .aux file, and the next run reads it. A value written in run p is first readable in run p+1, so the pass lag is exactly one. Generated lists and bibliographies add more steps. Deployed tools stop with five different tests and four fixed limits: latexmk 5, Tectonic 6, ConTeXt 9, and Typst 5. latexrun documents no limit.

Problem statement

No deployed tool derives the run count from the source. A fixed schedule is wrong for documents that need more runs than the schedule allows. The engine rerun warning misses changes in generated lists. A document can also change its own dependency graph: a page reference can change the page count, and the page count decides how many page references exist. Classical pass numbering assumes a fixed graph, so it does not apply to every document.

Objectives

- Classify a document before the first engine run and name the facts that cause a non-convergent result, with source locations.
- Derive a run-count bound for documents that can be certified.

- Plan the build from the class and verify the result against the real engine.
- Measure the analysis against five deployed stopping tests on a public corpus, including the cases where it fails.

4. Literature Review

Work	What it contributes	Relation to AuxFlow
Bochmann 1976; Alblas 1981	Static pass numbering for attribute grammars. Alblas separates simple from pure multi-pass evaluation. Minimum pass assignment is NP-complete (Raiha and Ukkonen 1981).	AuxFlow numbers fact instances, which is the pure setting. It computes a conservative bound and makes no minimality claim.
Farrow 1986; Jones 1990; Magnusson and Hedin 2007	Circular attribute grammars. Jones splits the dependency graph into strongly connected components. Magnusson and Hedin state the convergence condition (finite height, monotone functions) and note that their evaluator does not check it.	AuxFlow checks the finite-height half in a restricted domain, using a static influence analysis.
Hecht and Ullman 1975; Kam and Ullman 1976	Iterative data-flow rounds are bounded by a depth parameter of the graph.	The AuxFlow plan has the same shape: P runs to propagate, one run to confirm.
Mokhov, Mitchell and Peyton Jones 2018; Erdweg et al. 2015 (Pluto)	Static and dynamic dependencies in build systems. Pluto builds LaTeX with a fixpoint cycle handler and judges static dependency detection for LaTeX to be a lost cause.	AuxFlow answers with a declared subset. Constructs outside it become opaque facts and make the document UNKNOWN.
latexmk, latexrun, Tectonic, ConTeXt, Typst; Gaspar 2014; Madje 2022	Five stopping tests and five limits. Gaspar gives a document that never settles. Madje shows the stability test is undecidable for arbitrary code and proposes a cap with an error at the guilty call.	These tools are the baselines. Gaspar's loop is corpus member b08.

AuxFlow does not claim to invent pass numbering, strongly connected component analysis, or checksum verification. The new part is the combination: a static influence test from fact values to constructs that control content volume, and a controlled comparison of the deployed stopping tests.

5. Methodology

Proposed system and architecture

The pipeline has a static part and a dynamic part. Everything up to the build plan runs without a TeX engine. Only auxflow run starts a subprocess.

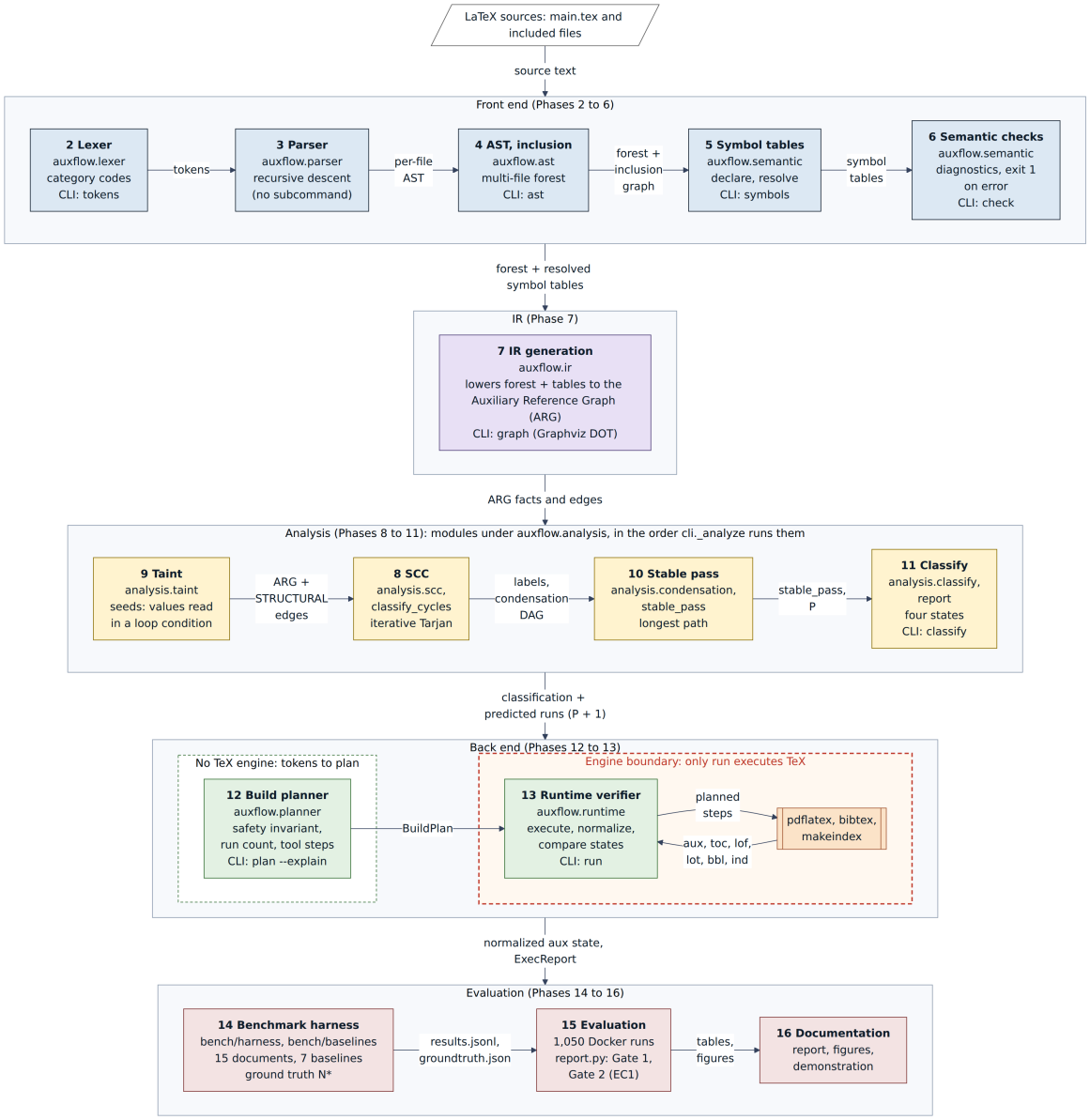


Figure 1. AuxFlow compiler pipeline

Data model

The Auxiliary Reference Graph (ARG) has one node for each fact instance and one edge for each dependency. A fact has a kind, a key, a component (exists or value), and a mandatory source span. An edge has a pass lag, a type (AUX_ROUNDTRIP, TOOL, LAYOUT, or STRUCTURAL), and a reason string.

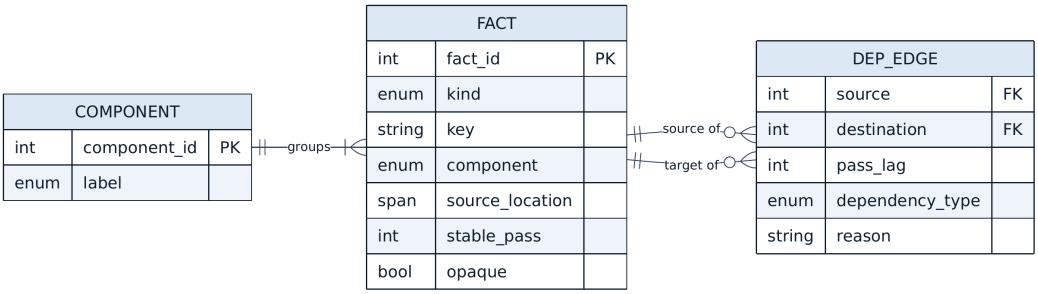


Figure 2. Entity-relationship model of the ARG

Analysis workflow

- Taint analysis seeds the facts that a loop bound or condition reads, then propagates along value edges. Each tainted fact gets a STRUCTURAL edge to the content-volume fact. The cost is O(V + E).
- An iterative Tarjan algorithm finds the strongly connected components. A component with an internal STRUCTURAL edge is AMPLIFYING. A component with an opaque fact is UNKNOWN_CYCLE. Any other cycle is BENIGN.
- Stable-pass numbering follows the longest path on the condensation. For a fact f, s(f) = 1 if f has no dependencies, and otherwise 1 + the largest s(g) + lag(g to f) - 1 over its dependencies g. P is the largest s(f) over facts stored in auxiliary files, and the predicted run count is N = P + 1.

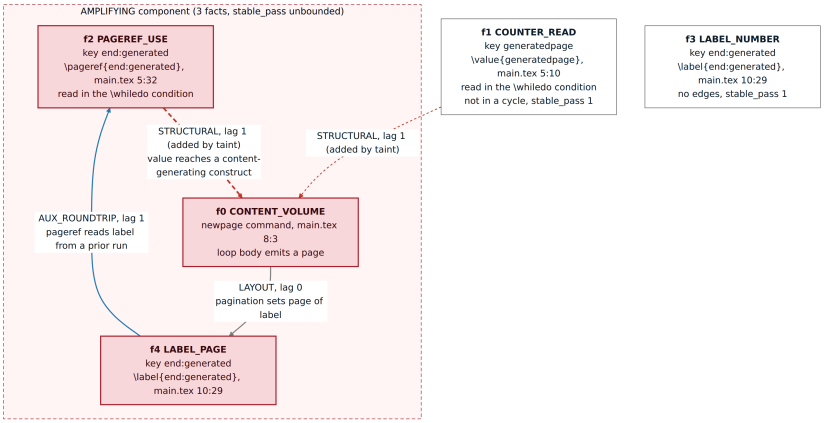


Figure 3. ARG of b08_cycle. Taint analysis adds the STRUCTURAL edge that closes the AMPLIFYING cycle.

Classification and planning

Only a CERTIFIED plan uses the derived run count. Every other executable plan has at least three engine runs, and an assertion in the planner enforces this. An AMPLIFYING document gets no plan unless the user passes --force, which runs it under a cap of six runs.

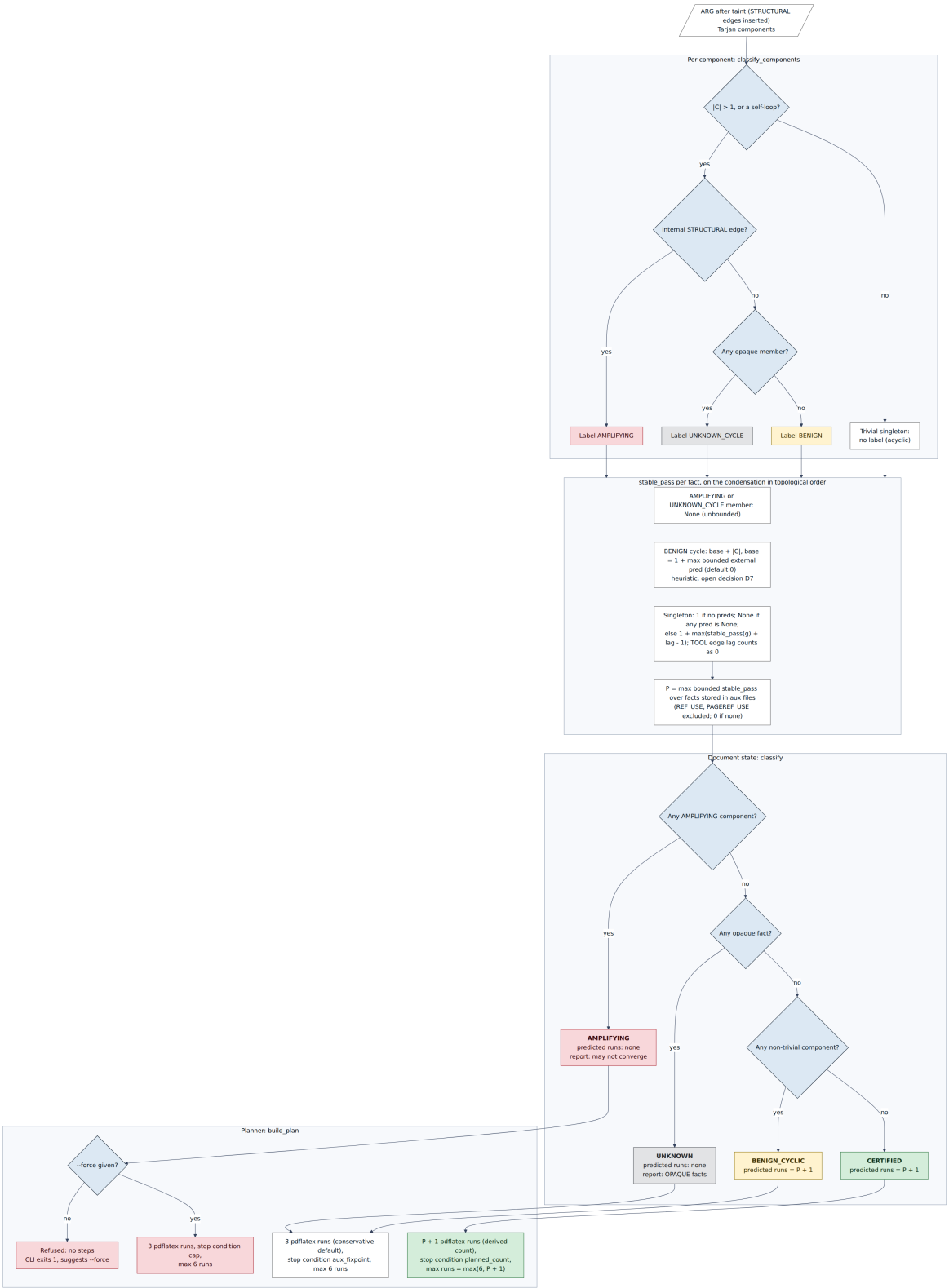


Figure 4. Four-state classifier and the plan for each state

Runtime verification

After each engine run the verifier parses the auxiliary files into a set of canonical records and compares it with the set from the previous run. It does not use engine warnings. If the plan ends while the state still changes, it repeats the last engine step up to the cap and records a prediction miss when a certified document needs more runs than predicted.

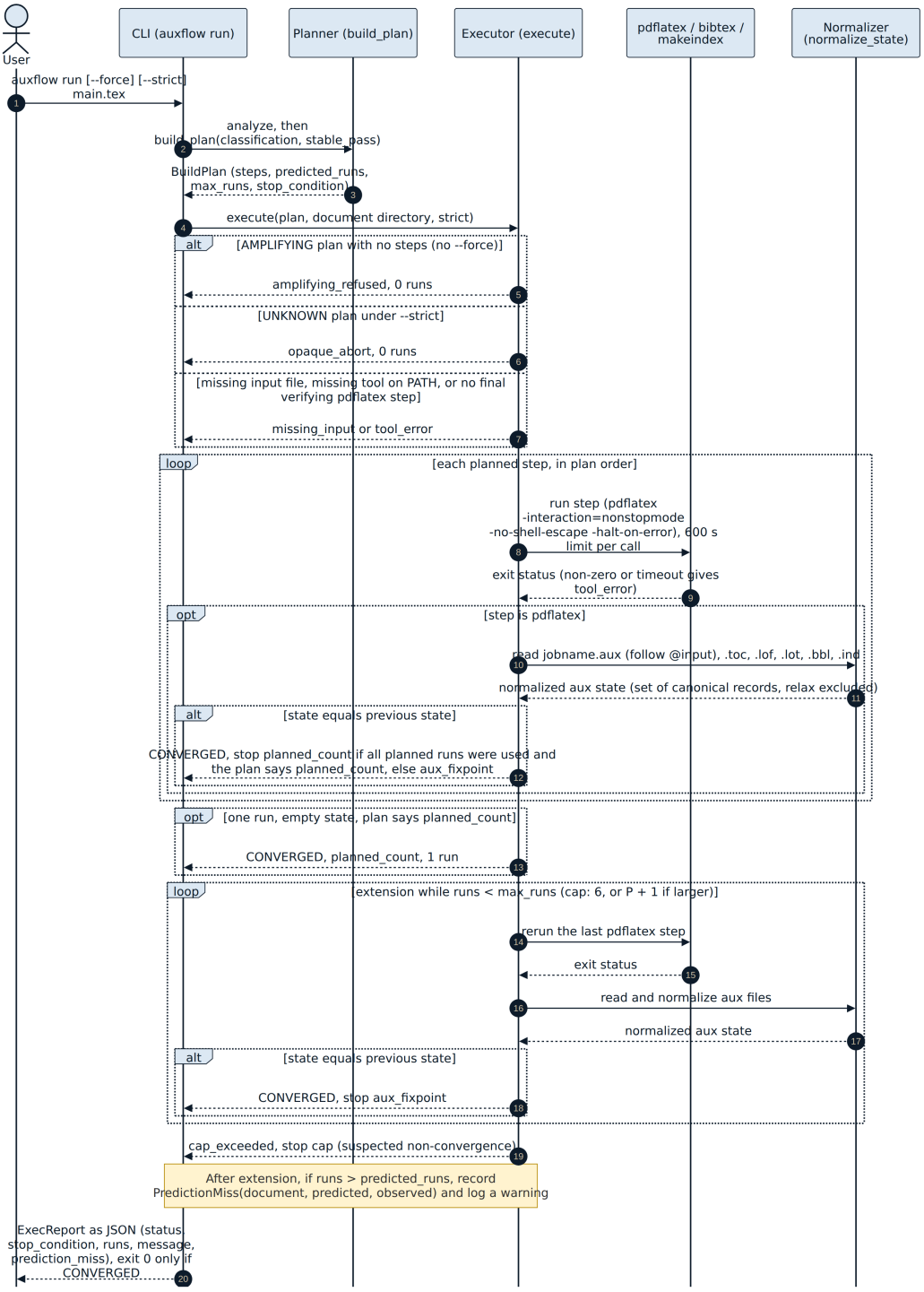


Figure 5. auxflow run: plan execution with runtime verification

Tools

Python 3.12 with no third-party runtime dependencies. pytest and Hypothesis for tests, ruff and mypy --strict for quality checks, Docker for the pinned benchmark image, and hatch-vcs for versioning from git tags.

6. Implementation

The project was built in 16 phases. Review 2 covered phases 1 to 7 (lexer, parser, syntax tree, symbol tables, semantic checks, and the ARG). This review covers phases 8 to 16.

Phase	Module	Function
8	auxflow.analysis.scc	Iterative Tarjan SCC and cycle labels
9	auxflow.analysis.taint	Seed and propagate taint, add STRUCTURAL edges
10	auxflow.analysis.stable_pass	Stable-pass number per fact, N = P + 1
11	auxflow.analysis.classify	Four-state classifier and diagnostics
12	auxflow.planner	Build plan with the safety rule and the --force cap
13	auxflow.runtime	State normalisation, fixpoint check, auxflow run
14 to 16	bench/	Corpus, 7 baselines, ground truth, evaluation report

Changes after the first measurement

- A guard makes a page reference under roman, Roman, alph, or Alph page numbering UNKNOWN, because the numeral width depends on its value. \vref and \vpageref also make a document UNKNOWN.
- A list of inert packages and commands keeps ordinary front matter from making a document UNKNOWN.
- auxflow run limits each tool call to 600 seconds, passes -no-shell-escape, and rejects auxiliary files outside the work directory.
- auxflow update uses pipx when the environment has no pip.

Code and quality figures

- About 5,600 lines of source code in 47 modules and about 5,900 lines of test code in 57 files.
- 396 tests pass and 1 is skipped. mypy --strict reports no issues in 48 source files. ruff reports no issues in src and tests.

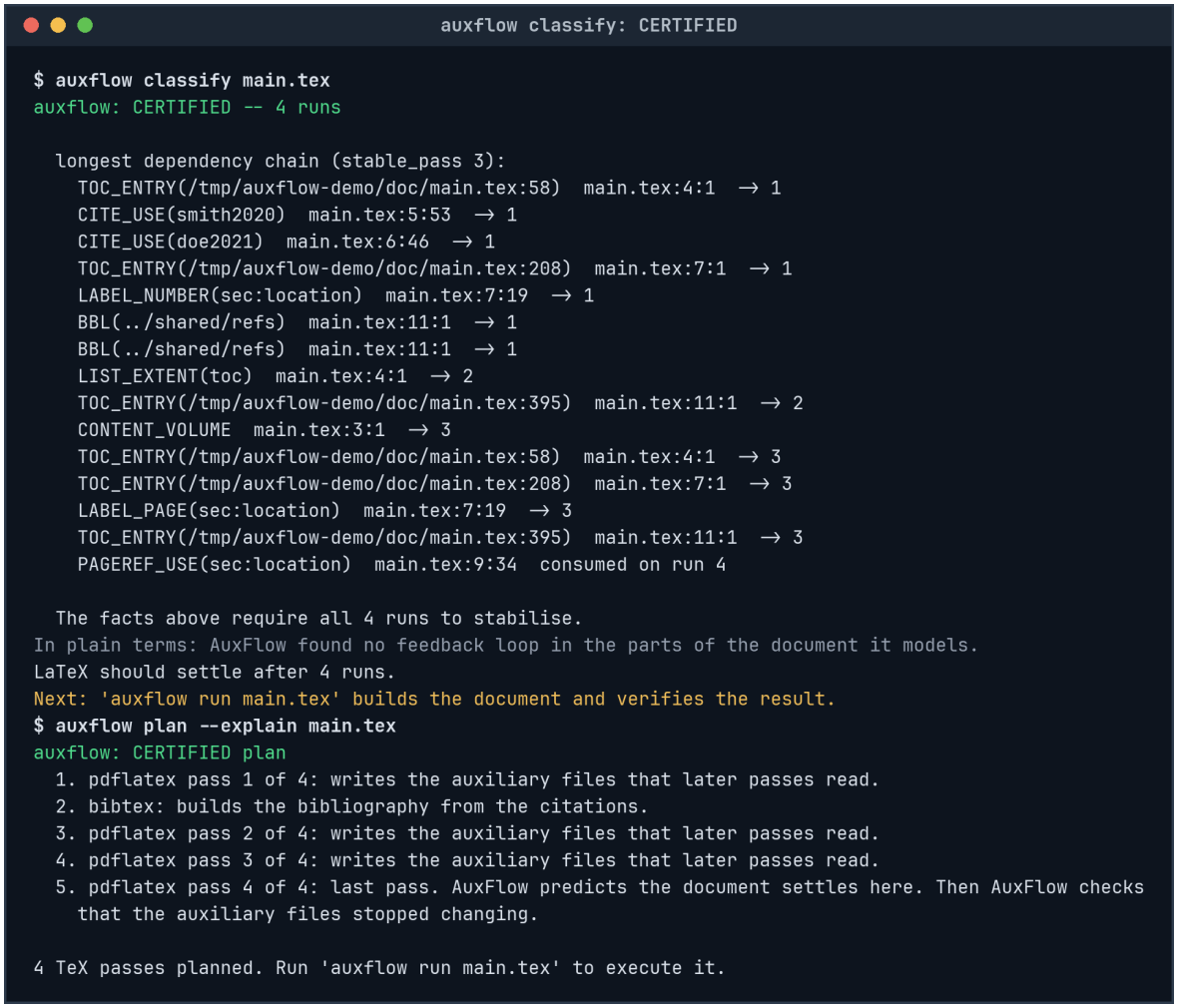


Figure 6. auxflow classify and plan on a CERTIFIED document

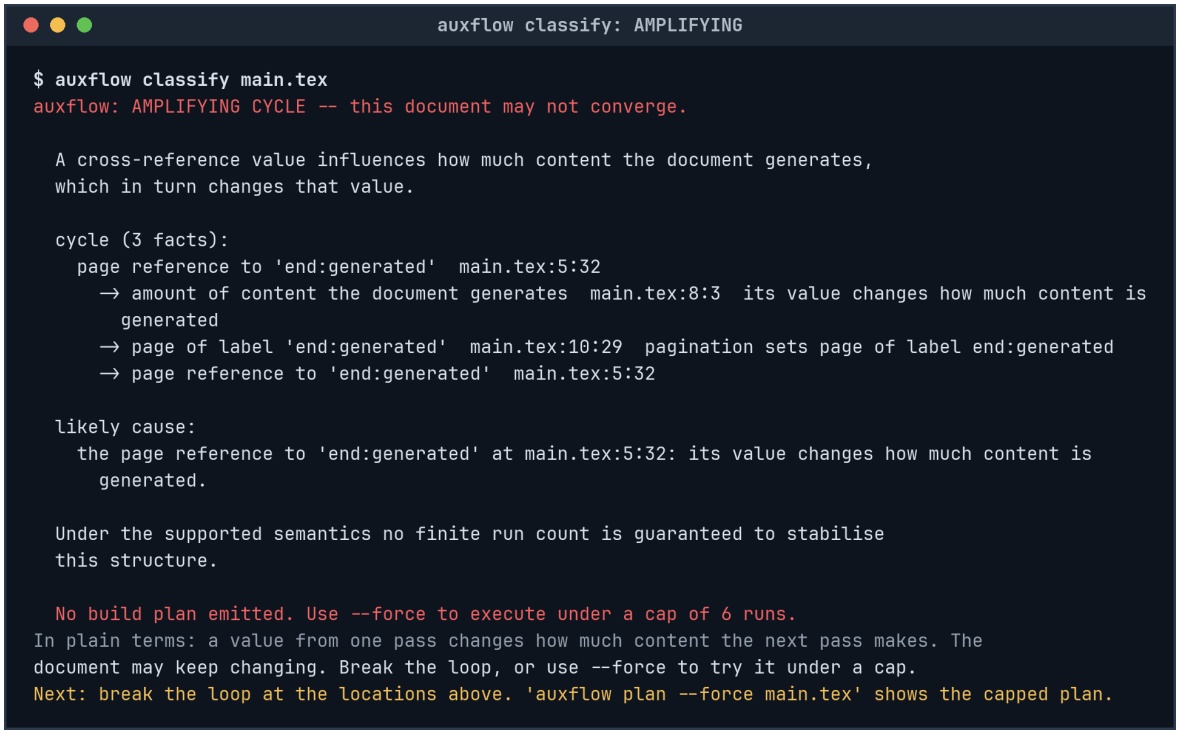


Figure 7. auxflow classify on an AMPLIFYING document

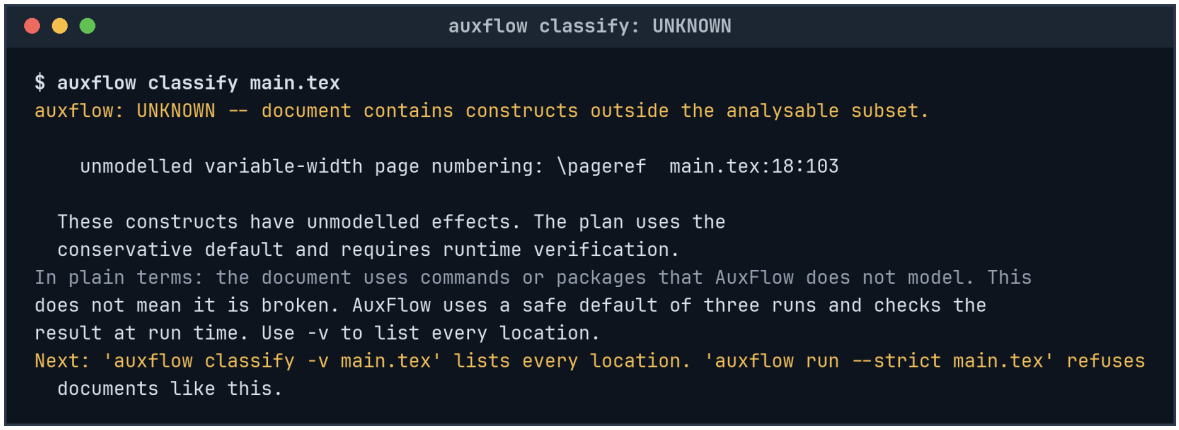


Figure 8. auxflow classify on an UNKNOWN document

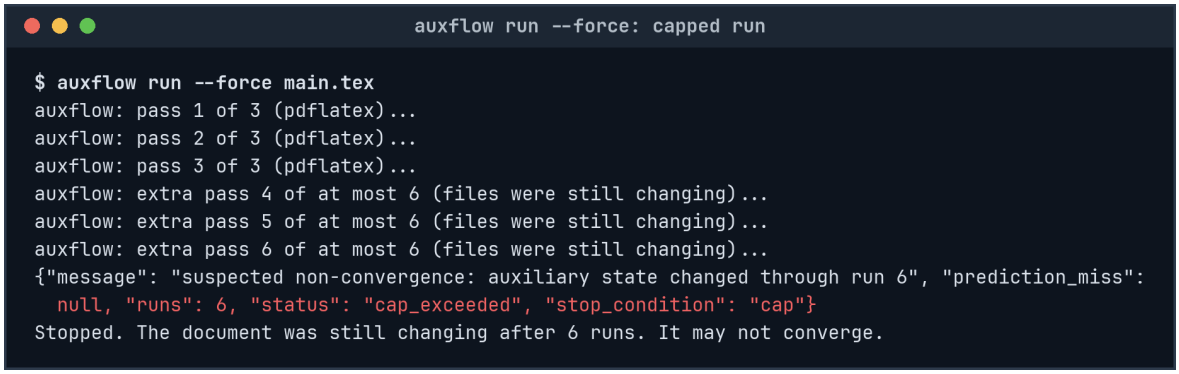


Figure 9. auxflow run --force stops at the cap of six runs

7. Results and Discussion

Experimental setup

The corpus has 15 documents (b01_plain to b15_ibid) that cover the non-convergence mechanisms M1 to M9, including a multi-chapter thesis. Seven baselines ran ten times on each document, one container per repetition, with one CPU and no network: 1,050 runs. A is a fixed three-run schedule, B the rerun warning, C output comparison, C' input comparison, C'' read-then-written comparison, D AuxFlow static analysis, and E AuxFlow with runtime verification. Ground truth N* comes from eight unconditional engine passes and is independent of every baseline. The expected class of each document was written before the analyzer.

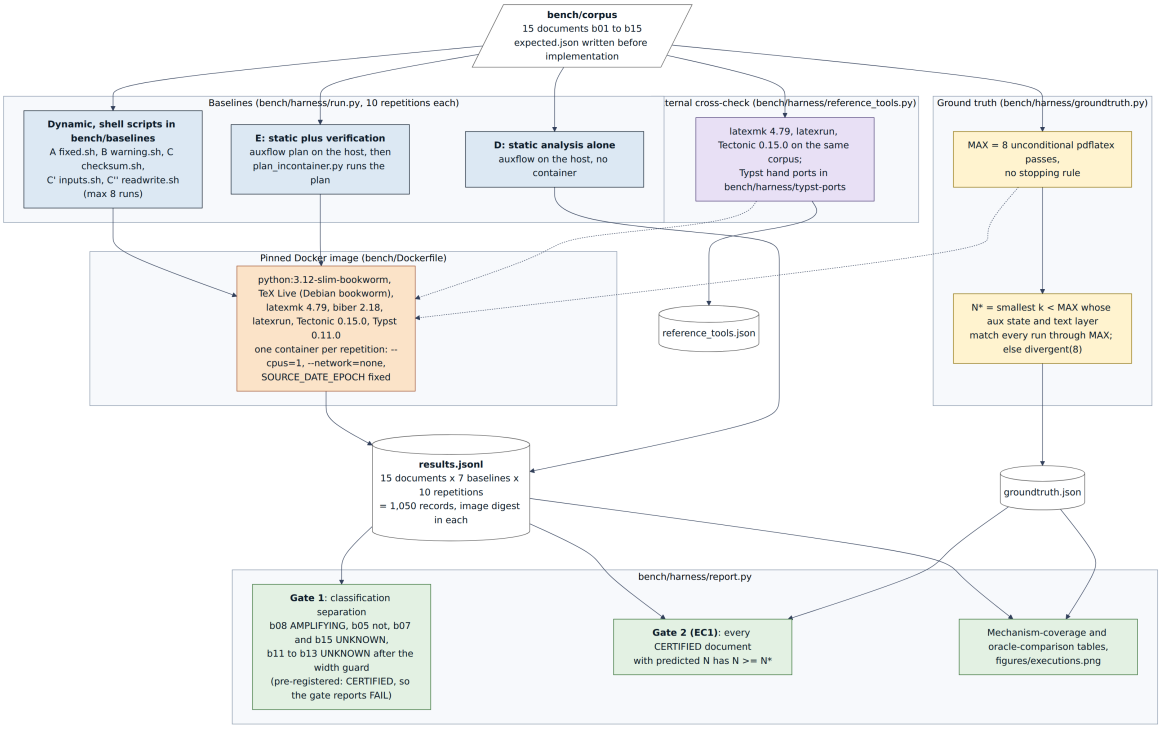


Figure 10. Evaluation setup: corpus, baselines, ground truth, and gates

Classification and run-count bound

Document	N*	Before the guard	Current class	Predicted N
b01_plain	1	CERTIFIED	CERTIFIED	1
b02_backref	2	CERTIFIED	CERTIFIED	2
b03_fwdref	2	CERTIFIED	CERTIFIED	2
b04_chain	2	CERTIFIED	CERTIFIED	3
b05_toc	3	CERTIFIED	CERTIFIED	3
b06_bib	3	CERTIFIED	CERTIFIED	4
b07_footnote	2	UNKNOWN	UNKNOWN	none
b08_cycle	divergent	AMPLIFYING	AMPLIFYING	none
b09_thesis	3	CERTIFIED	CERTIFIED	4

b10_random	divergent	UNKNOWN	UNKNOWN	none
b11_width	oscillating	CERTIFIED	UNKNOWN	none
b12_float	oscillating	CERTIFIED	UNKNOWN	none
b13_varioref	oscillating	CERTIFIED	UNKNOWN	none
b14_ifthen	oscillating	AMPLIFYING	AMPLIFYING	none
b15_ibid	3	UNKNOWN	UNKNOWN	none

- All 7 certified documents satisfy N >= N*. The bound is exact for 4 and larger by one for 3 (b04, b06, b09). No bound is too small.
- b08 is AMPLIFYING and b05 is not. These two checks guard against an analysis that says nothing, and both pass.
- All 6 non-convergent documents are flagged before the engine starts: 2 AMPLIFYING and 4 UNKNOWN. None is certified.
- b14 is AMPLIFYING by over-approximation. Nothing in it changes the quantity of content, only its size. Typst compiles the ported document without an error.

Gate 1 is reported as FAIL

The pre-registered expectation for b11, b12, and b13 was CERTIFIED, as declared false negatives. The guard described in section 6 changed all three to UNKNOWN, so bench/harness/report.py reports Gate 1 as FAIL for these three rows. Gate 2 (N >= N* for every certified document) passes with no violation. The guard was added after the first measurement, so the corpus does not test it independently. It matches syntax, not the mechanism. Width feedback under arabic numbering is not covered, and the analysis would probably certify such a document.

Stopping tests compared

Outcome	A	B	C	C'	C''	E	D
Correct stop, of 9 convergent documents	9	7	9	9	9	7	7
Premature stop on a convergent document	0	2	0	0	0	0	0
Withheld by static refusal	0	0	0	0	0	2	2
Non-convergence flagged, of 6	0	5	6	6	6	6	6
Non-convergence accepted as	6	1	0	0	0	0	0

converged							
Engine runs on the 9 convergent documents	27	17	22	22	22	17	0

- Baseline A accepts all six non-convergent documents after three runs.
- Baseline B is unsound in both directions. It accepts b10 after one run in all ten repetitions, although b10 never settles. It stops after one run on b05 and b15, where N* is 3.
- C, C', and C'' agree on every document. The expected one-run gap between output comparison and input comparison did not appear on this corpus.
- E spends 17 runs on the 7 documents it executes, against 21 for A on the same documents. It refuses 8 documents before the first run: the 6 non-convergent ones, and b07 and b15, which contain opaque constructs.
- b01_plain needs two runs under E although N* is 1. pdfTeX writes a page-count record to every .aux file, so the verifier needs a second run to see two equal states. It records a prediction miss in all ten repetitions. The prediction is right.

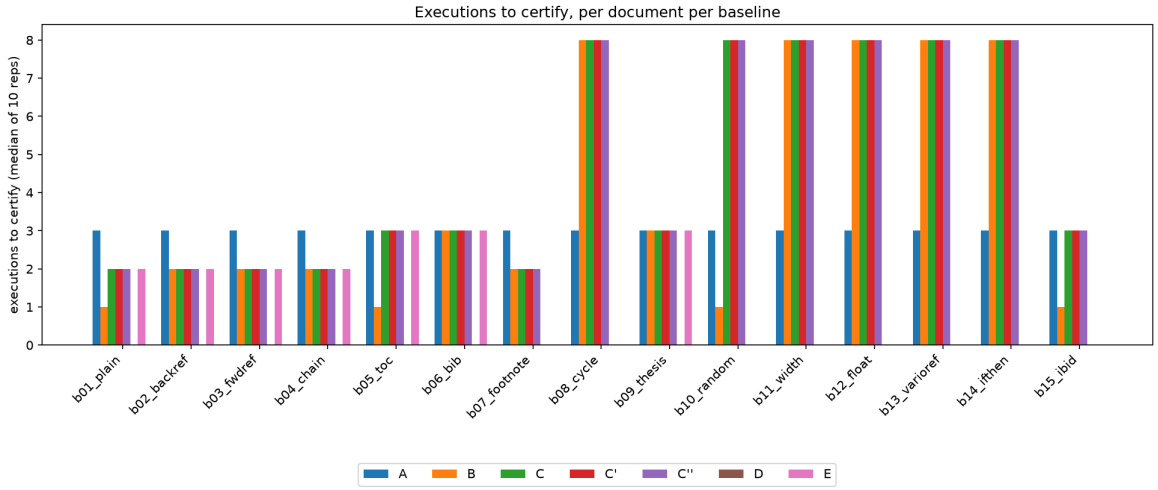


Figure 11. Engine runs until a verdict, per document and baseline (generated by bench/harness/report.py)

External tools

latexmk and latexrun fail to converge on all six non-convergent documents, so these documents are real non-convergence and not artifacts of the corpus. latexrun also fails on b15, which latexmk accepts. Tectonic compiled only b01 and b08 offline. On b08 it stops at its limit of six passes. Typst rejects a hand port of b08 with 'loop seems to be infinite'.

Cost

Static analysis takes 1.3 ms at the median and 10.4 ms at most. The largest graph is b09_thesis with 91 facts and 73 edges, at 7.5 ms. This time is reported as overhead and is not subtracted from any run time.

Threats to validity

- The corpus has 15 documents and most are synthetic. All runs used one machine.
- One person wrote the corpus, the expectations, and the analyzer. The expectations came first and are in the repository.
- The model assumes that resolving a reference does not change pagination. This is false in general, and the analysis is unsound for the extent-driven class.
- The container holds TeX Live 2022. Run counts may differ on other releases. A leftover .aux file in a corpus directory once made three baselines stop early on b05. Those records were removed and run again.

8. Conclusion and Future Work

Outcomes

- AuxFlow classifies a document before any engine run. On the corpus it flagged all 6 non-convergent documents and bounded all 7 certified documents correctly.
- Static analysis takes about a millisecond, and the verifier catches a wrong prediction at run time.
- The measured comparison shows where the deployed stopping tests disagree: the fixed schedule and the rerun warning accept documents that never settle.
- The research paper is written. It is in paper/ and is linked from the project site.

Limitations

- The analysis covers a declared subset. Anything outside it makes a document UNKNOWN.
- It cannot see most extent-driven feedback. The width guard is syntactic.
- The AMPLIFYING label joins divergence and oscillation. The bound for a benign cycle is a heuristic, and the corpus has no benign cyclic member.

Improvements

- Test on a larger set of real documents.
- Replace the syntactic width guard with a test that does not depend on syntax.
- Split AMPLIFYING into divergent and oscillating cases.
- Model more packages, so fewer documents are UNKNOWN, and prove a bound for benign cycles.

9. References

- G. V. Bochmann, Semantic evaluation from left to right, Communications of the ACM 19(2):55-62, 1976.
- H. Alblas, A characterization of attribute evaluation in passes, Acta Informatica 16(4):427-464, 1981.
- K.-J. Raiha and E. Ukkonen, Minimizing the number of evaluation passes for attribute grammars, SIAM Journal on Computing 10(4):772-786, 1981.
- R. Farrow, Automatic generation of fixed-point-finding evaluators for circular, but well-defined, attribute grammars, SIGPLAN Notices 21(7):85-98, 1986.
- L. G. Jones, Efficient evaluation of circular attribute grammars, ACM TOPLAS 12(3):429-462, 1990.
- E. Magnusson and G. Hedin, Circular reference attributed grammars: their evaluation and applications, Science of Computer Programming 68(1):21-37, 2007.
- M. S. Hecht and J. D. Ullman, A simple algorithm for global data flow analysis problems, SIAM Journal on Computing 4(4):519-532, 1975.
- J. B. Kam and J. D. Ullman, Global data flow analysis and iterative algorithms, Journal of the ACM 23(1):158-171, 1976.
- A. Mokhov, N. Mitchell, and S. Peyton Jones, Build systems a la carte, Proceedings of the ACM on Programming Languages 2(ICFP), 2018.
- S. Erdweg, M. Lichter, and M. Weiel, A sound and optimal incremental build system with dynamic dependencies, OOPSLA 2015.
- R. Tarjan, Depth-first search and linear graph algorithms, SIAM Journal on Computing 1(2):146-160, 1972.
- Gaspar, Does not suffice to run LaTeX a finite number of times to get cross-references right, TUGboat 35(1):16, 2014.
- L. Madje, A Programmable Markup Language for Typesetting, MSc thesis, Technische Universitat Berlin, 2022.
- J. Collins, latexmk, https://ctan.org/pkg/latexmk (version 4.88).
- A. Clements, latexrun, https://github.com/aclements/latexrun.
- Tectonic, https://github.com/tectonic-typesetting/tectonic (version 0.15.0).
- Typst, https://github.com/typst/typst (version 0.11.0).
- AuxFlow source, corpus, harness, and results, https://github.com/upayanmazumder/auxflow, and PyPI package auxflow.
- U. Mazumder and B. R., AuxFlow: A static applicability test for fixpoint convergence in multi-pass LaTeX builds, 2026, paper/main.pdf in the repository.